

```
from google.colab import files
files.upload()
```

Show hidden output

```
!unzip resume_parser.zip
%cd resume_parser
```

Show hidden output

```
!pip install transformers datasets accelerate python-docx pypdf seqeval safetensors
```

Show hidden output

```
!python train.py
```

```
{'loss': '1.613e-05', 'grad_norm': '0.0001869', 'learning_rate': '5.168e-06', 'epoch': '2.225'}
{'loss': '1.566e-05', 'grad_norm': '0.0007631', 'learning_rate': '5.126e-06', 'epoch': '2.231'}
{'loss': '1.539e-05', 'grad_norm': '0.0002736', 'learning_rate': '5.084e-06', 'epoch': '2.237'}
{'loss': '1.58e-05', 'grad_norm': '0.0003544', 'learning_rate': '5.043e-06', 'epoch': '2.244'}
{'loss': '1.595e-05', 'grad_norm': '0.0003111', 'learning_rate': '5.001e-06', 'epoch': '2.25'}
{'loss': '1.502e-05', 'grad_norm': '0.0001744', 'learning_rate': '4.959e-06', 'epoch': '2.256'}
{'loss': '1.685e-05', 'grad_norm': '0.0002803', 'learning_rate': '4.918e-06', 'epoch': '2.263'}
{'loss': '1.5e-05', 'grad_norm': '0.0003911', 'learning_rate': '4.876e-06', 'epoch': '2.269'}
{'loss': '1.549e-05', 'grad_norm': '0.00121', 'learning_rate': '4.834e-06', 'epoch': '2.275'}
{'loss': '1.437e-05', 'grad_norm': '0.0002827', 'learning_rate': '4.793e-06', 'epoch': '2.281'}
{'loss': '1.461e-05', 'grad_norm': '0.0002007', 'learning_rate': '4.751e-06', 'epoch': '2.288'}
{'loss': '1.451e-05', 'grad_norm': '0.0004342', 'learning_rate': '4.709e-06', 'epoch': '2.294'}
{'loss': '1.517e-05', 'grad_norm': '0.0001893', 'learning_rate': '4.668e-06', 'epoch': '2.3'}
{'loss': '1.437e-05', 'grad_norm': '0.0004271', 'learning_rate': '4.626e-06', 'epoch': '2.306'}
{'loss': '1.2905e-05', 'grad_norm': '0.00105', 'learning_rate': '4.584e-06', 'epoch': '2.312'}
{'loss': '1.363e-05', 'grad_norm': '0.0001259', 'learning_rate': '4.543e-06', 'epoch': '2.319'}
{'loss': '1.458e-05', 'grad_norm': '0.0002371', 'learning_rate': '4.501e-06', 'epoch': '2.325'}
{'loss': '1.413e-05', 'grad_norm': '0.0003699', 'learning_rate': '4.459e-06', 'epoch': '2.331'}
{'loss': '1.532e-05', 'grad_norm': '0.0002466', 'learning_rate': '4.418e-06', 'epoch': '2.337'}
{'loss': '1.386e-05', 'grad_norm': '0.0002407', 'learning_rate': '4.376e-06', 'epoch': '2.344'}
{'loss': '1.353e-05', 'grad_norm': '0.0003277', 'learning_rate': '4.334e-06', 'epoch': '2.35'}
{'loss': '1.404e-05', 'grad_norm': '0.001124', 'learning_rate': '4.293e-06', 'epoch': '2.356'}
{'loss': '1.321e-05', 'grad_norm': '0.0002369', 'learning_rate': '4.251e-06', 'epoch': '2.362'}
{'loss': '1.283e-05', 'grad_norm': '0.0004527', 'learning_rate': '4.209e-06', 'epoch': '2.369'}
{'loss': '1.357e-05', 'grad_norm': '0.000364', 'learning_rate': '4.168e-06', 'epoch': '2.375'}
{'loss': '1.342e-05', 'grad_norm': '0.000436', 'learning_rate': '4.126e-06', 'epoch': '2.381'}
{'loss': '1.474e-05', 'grad_norm': '0.0004168', 'learning_rate': '4.084e-06', 'epoch': '2.388'}
{'loss': '1.168e-05', 'grad_norm': '0.0004398', 'learning_rate': '4.043e-06', 'epoch': '2.394'}
{'loss': '1.319e-05', 'grad_norm': '0.0004109', 'learning_rate': '4.001e-06', 'epoch': '2.4'}
{'loss': '1.243e-05', 'grad_norm': '0.0002782', 'learning_rate': '3.959e-06', 'epoch': '2.406'}
{'loss': '1.238e-05', 'grad_norm': '0.0002399', 'learning_rate': '3.918e-06', 'epoch': '2.413'}
{'loss': '1.201e-05', 'grad_norm': '0.0002568', 'learning_rate': '3.876e-06', 'epoch': '2.419'}
{'loss': '1.204e-05', 'grad_norm': '0.0001462', 'learning_rate': '3.834e-06', 'epoch': '2.425'}
{'loss': '1.23e-05', 'grad_norm': '0.0003564', 'learning_rate': '3.793e-06', 'epoch': '2.431'}
{'loss': '1.231e-05', 'grad_norm': '0.0005253', 'learning_rate': '3.751e-06', 'epoch': '2.438'}
{'loss': '1.23e-05', 'grad_norm': '0.0001967', 'learning_rate': '3.709e-06', 'epoch': '2.444'}
{'loss': '1.177e-05', 'grad_norm': '0.0002765', 'learning_rate': '3.668e-06', 'epoch': '2.45'}
{'loss': '1.222e-05', 'grad_norm': '0.0002337', 'learning_rate': '3.626e-06', 'epoch': '2.456'}
{'loss': '1.15e-05', 'grad_norm': '0.0002374', 'learning_rate': '3.584e-06', 'epoch': '2.462'}
{'loss': '1.074e-05', 'grad_norm': '0.0001065', 'learning_rate': '3.543e-06', 'epoch': '2.469'}
{'loss': '1.107e-05', 'grad_norm': '0.000157', 'learning_rate': '3.501e-06', 'epoch': '2.475'}
{'loss': '1.096e-05', 'grad_norm': '0.0002087', 'learning_rate': '3.459e-06', 'epoch': '2.481'}
{'loss': '1.109e-05', 'grad_norm': '0.0002148', 'learning_rate': '3.418e-06', 'epoch': '2.487'}
{'loss': '1.183e-05', 'grad_norm': '0.000161', 'learning_rate': '3.376e-06', 'epoch': '2.494'}
{'loss': '1.057e-05', 'grad_norm': '0.0002551', 'learning_rate': '3.334e-06', 'epoch': '2.5'}
{'loss': '1.18e-05', 'grad_norm': '0.0004215', 'learning_rate': '3.293e-06', 'epoch': '2.506'}
{'loss': '1.107e-05', 'grad_norm': '0.0003138', 'learning_rate': '3.251e-06', 'epoch': '2.513'}
{'loss': '1.019e-05', 'grad_norm': '0.0001994', 'learning_rate': '3.209e-06', 'epoch': '2.519'}
{'loss': '1.063e-05', 'grad_norm': '0.0003391', 'learning_rate': '3.168e-06', 'epoch': '2.525'}
{'loss': '1.03e-05', 'grad_norm': '0.0001714', 'learning_rate': '3.126e-06', 'epoch': '2.531'}
{'loss': '1.014e-05', 'grad_norm': '0.0002004', 'learning_rate': '3.084e-06', 'epoch': '2.538'}
{'loss': '9.523e-06', 'grad_norm': '0.0003797', 'learning_rate': '3.043e-06', 'epoch': '2.544'}
{'loss': '1.017e-05', 'grad_norm': '0.0001801', 'learning_rate': '3.001e-06', 'epoch': '2.55'}
{'loss': '0.02228', 'grad_norm': '0.0001058', 'learning_rate': '2.959e-06', 'epoch': '2.556'}
{'loss': '1.161e-05', 'grad_norm': '0.0003019', 'learning_rate': '2.918e-06', 'epoch': '2.562'}
{'loss': '8.612e-05', 'grad_norm': '0.0001344', 'learning_rate': '2.876e-06', 'epoch': '2.569'}
{'loss': '1.125e-05', 'grad_norm': '0.0001266', 'learning_rate': '2.834e-06', 'epoch': '2.575'}
{'loss': '1.043e-05', 'grad_norm': '0.0002771', 'learning_rate': '2.793e-06', 'epoch': '2.581'}
```

```
from google.colab import files
files.download('model/model.safetensors')
files.download('model/tokenizer.json')
files.download('model/tokenizer_config.json')
files.download('model/training_args.bin')
files.download('model/eval_results.json')
```

```

!pip install matplotlib seaborn pandas numpy

import json
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
from collections import Counter, defaultdict

plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['font.size'] = 11
sns.set_theme(style="whitegrid")

# Upload eval_results.json
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]

with open(file_name, 'r') as f:
    results = json.load(f)

test = results['test']
val = results['val']
training_loss = results['training_loss']
dataset_stats = results['dataset_stats']

print("✅ Results loaded!")
print(f"Test Item F1: {test['item_f1']}")
print(f"Test Section F1: {test['section_f1']}")

```

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving eval_results.json to eval_results.json
✅ Results loaded!
Test Item F1: 0.9709

```

# Count B- tag occurrences in training data (excluding O)
item_counts = Counter()
with open('data/train_all.txt', 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if line:
            parts = line.split()
            if len(parts) >= 2:
                tag = parts[1]
                if tag != 'O':
                    base = tag[2:] if tag.startswith(('B-', 'I-')) else tag
                    item_counts[base] += 1

entities = sorted(item_counts.keys(), key=lambda x: item_counts[x], reverse=True)
counts = [item_counts[e] for e in entities]

fig, ax = plt.subplots(figsize=(12, 6))
colors = sns.color_palette("viridis", len(entities))
bars = ax.bar(range(len(entities)), counts, color=colors, edgecolor='white', linewidth=0.5)

ax.set_xticks(range(len(entities)))
ax.set_xticklabels(entities, rotation=45, ha='right', fontsize=11, fontweight='bold')
ax.set_ylabel('Frequency', fontweight='bold', fontsize=12)
ax.set_title('FIGURE 4. Item Level: Category Frequencies in the Annotated Set\n(excluding O)',
             fontweight='bold', fontsize=13, pad=15)

```

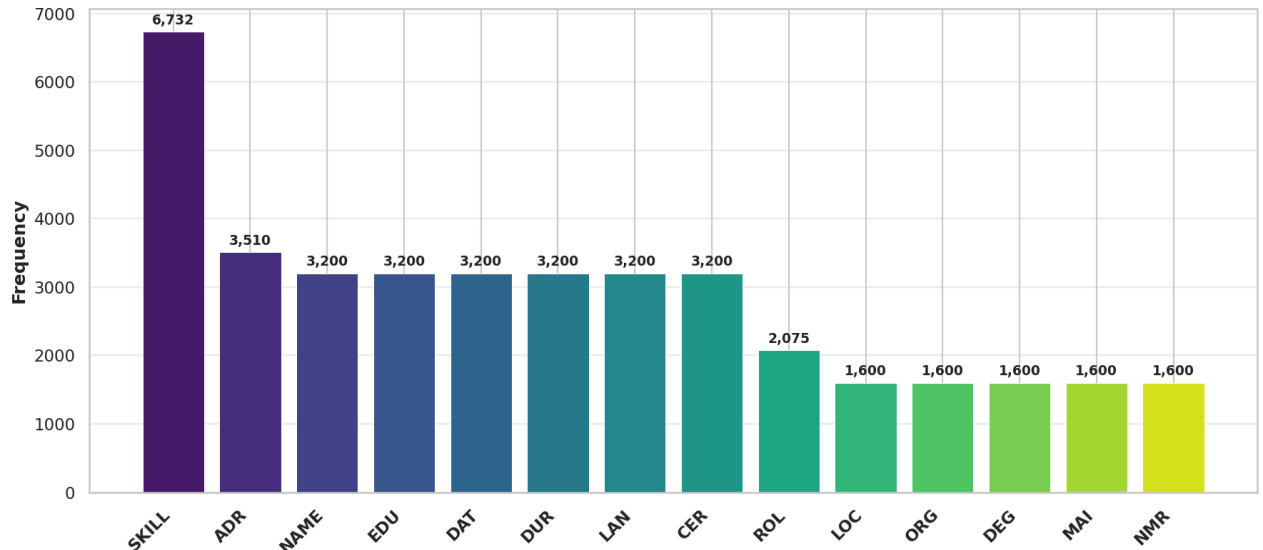
```

for i, v in enumerate(counts):
    ax.text(i, v + max(counts)*0.01, f'{v:,}', ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('fig4_item_category_freq.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig4_item_category_freq.png")

```

FIGURE 4. Item Level: Category Frequencies in the Annotated Set (excluding O)



✅ Saved: fig4_item_category_freq.png

```

section_counts = Counter()
with open('data/train_all.txt', 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if line:
            parts = line.split()
            if len(parts) >= 3:
                section = parts[2]
                section_counts[section] += 1

sections = sorted(section_counts.keys(), key=lambda x: section_counts[x], reverse=True)
sec_vals = [section_counts[s] for s in sections]

fig, ax = plt.subplots(figsize=(10, 6))
colors_sec = ['#6366f1', '#f59e0b', '#10b981', '#ef4444', '#8b5cf6', '#06b6d4']
bars = ax.bar(range(len(sections)), sec_vals, color=colors_sec[:len(sections)], edgecolor='white', linewidth=0.5)

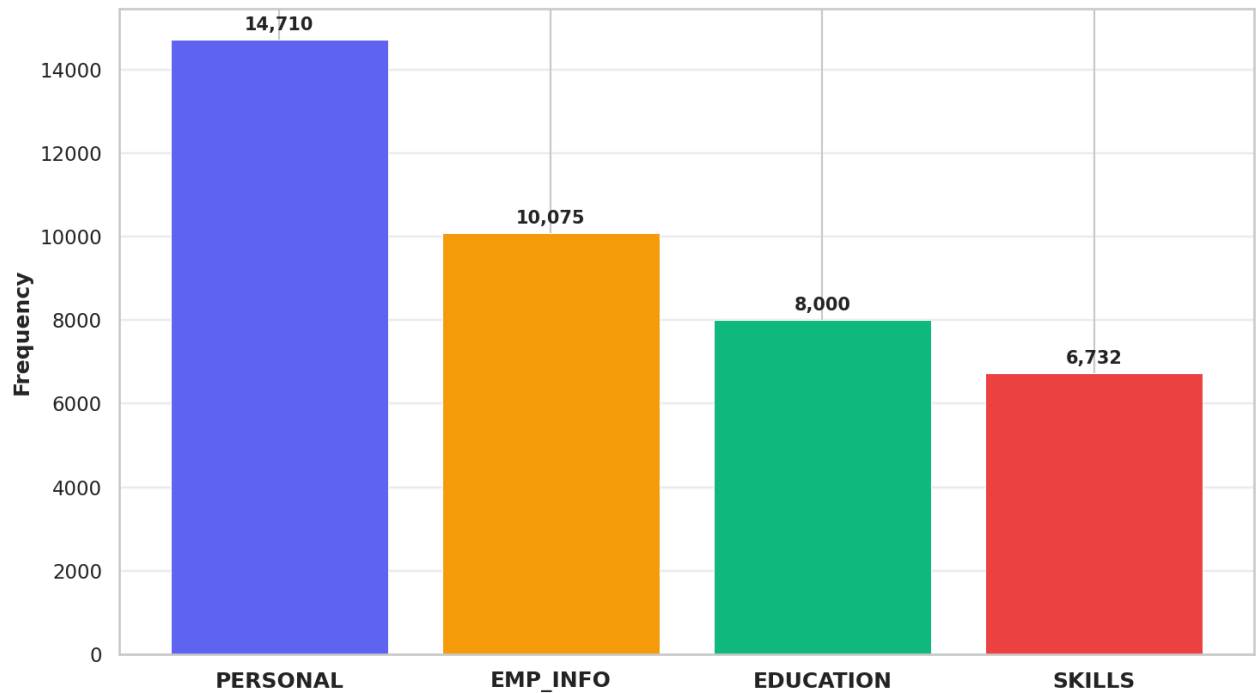
ax.set_xticks(range(len(sections)))
ax.set_xticklabels(sections, fontsize=12, fontweight='bold')
ax.set_ylabel('Frequency', fontweight='bold', fontsize=12)
ax.set_title('FIGURE 5. Section Level: Category Frequencies in the Annotated Set',
            fontweight='bold', fontsize=13, pad=15)

for i, v in enumerate(sec_vals):
    ax.text(i, v + max(sec_vals)*0.01, f'{v:,}', ha='center', va='bottom', fontsize=10, fontweight='bold')

ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('fig5_section_category_freq.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig5_section_category_freq.png")

```

FIGURE 5. Section Level: Category Frequencies in the Annotated Set



✓ Saved: fig5_section_category_freq.png

```
section_report = test['section_report']

rows = []
for section, m in section_report.items():
    if isinstance(m, dict) and 'f1-score' in m:
        rows.append({
            'Section': section,
            'Precision': f"{m['precision']:.4f}",
            'Recall': f"{m['recall']:.4f}",
            'F1-Score': f"{m['f1-score']:.4f}",
            'Support': int(float(m['support']))
        })

df = pd.DataFrame(rows)

fig, ax = plt.subplots(figsize=(10, max(3, len(df)*0.5 + 1)))
ax.axis('off')
ax.set_title('TABLE 1. Section Level of the Dual Model: Evaluation Results',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df.values, colLabels=df.columns, cellLoc='center', loc='center',
                 colColours=['#4f46e5']*len(df.columns))
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.6)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#4f46e5')
    else:
        cell.set_facecolor('#f8f9ff' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig('table1_section_eval.png', bbox_inches='tight', dpi=300)
```

```
plt.show()
print("✅ Saved: table1_section_eval.png")
```

TABLE 1. Section Level of the Dual Model: Evaluation Results

Section	Precision	Recall	F1-Score	Support
EDUCATION	0.9750	0.9488	0.9588	2000
EMP_INFO	0.9871	0.9709	0.9670	2539
PERSONAL	0.9730	0.9803	0.9772	3691
SKILLS	0.9518	0.9427	0.9558	1680
macro avg	0.9536	0.9510	0.9850	9910
weighted avg	0.9819	0.9558	0.9716	9910

```
✅ Saved: table1_section_eval.png
```

```
sec_macro = {
    'Metric': ['Precision', 'Recall', 'F1-Score'],
    'Test Set': [f"{test['section_precision']:.4f}", f"{test['section_recall']:.4f}", f"{test['section_f1']:.4f}"],
    'Val Set': [f"{val['section_precision']:.4f}", f"{val['section_recall']:.4f}", f"{val['section_f1']:.4f}"]
}
df_macro = pd.DataFrame(sec_macro)

fig, ax = plt.subplots(figsize=(8, 2.5))
ax.axis('off')
ax.set_title('TABLE 2. Section Level: Macro-Averaged Evaluation Results',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_macro.values, colLabels=df_macro.columns, cellLoc='center', loc='center',
                 colColours=['#f59e0b']*len(df_macro.columns))
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.8)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#f59e0b')
    else:
        cell.set_facecolor('#ffffeb' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig('table2_section_macro.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: table2_section_macro.png")
```

TABLE 2. Section Level: Macro-Averaged Evaluation Results

Metric	Test Set	Val Set
Precision	0.9713	0.9608
Recall	0.9695	0.9510
F1-Score	0.9491	0.9662

✔ Saved: table2_section_macro.png

```
lang_data = test['per_language']

rows_lang = []
for lang in ['English', 'Finnish', 'Swedish', 'Polish', 'Norwegian']:
    if lang in lang_data:
        m = lang_data[lang]
        rows_lang.append({
            'Language': lang,
            'Precision': f"{m['precision']:.4f}",
            'Recall': f"{m['recall']:.4f}",
            'F1-Score': f"{m['f1']:.4f}",
            'Samples': m['count']
        })

df_lang = pd.DataFrame(rows_lang)

fig, ax = plt.subplots(figsize=(10, 4))
ax.axis('off')
ax.set_title('TABLE 3. Section Level: Evaluation Results (All Languages)',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_lang.values, colLabels=df_lang.columns, cellLoc='center', loc='center',
                 colColours=['#10b981']*len(df_lang.columns))
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.6)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#10b981')
    else:
        cell.set_facecolor('#f0fdf4' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig('table3_section_all_lang.png', bbox_inches='tight', dpi=300)
plt.show()
print("✔ Saved: table3_section_all_lang.png")
```

TABLE 3. Section Level: Evaluation Results (All Languages)

Language	Precision	Recall	F1-Score	Samples
English	0.9836	0.9625	0.9596	2764
Finnish	0.9526	0.9672	0.9535	344
Swedish	0.9683	0.9829	0.9534	388
Polish	0.9514	0.9875	0.9597	307
Norwegian	0.9454	0.9434	0.9648	197

✔ Saved: table3_section_all_lang.png

```
item_report = test['item_report']

rows_item = []
for entity, m in item_report.items():
    if isinstance(m, dict) and 'f1-score' in m and entity not in ['micro avg', 'macro avg', 'weighted avg']:
        rows_item.append({
            'Entity': entity,
            'Precision': f"{m['precision']:.4f}",
            'Recall': f"{m['recall']:.4f}",
            'F1-Score': f"{m['f1-score']:.4f}",
            'Support': int(float(m['support']))
        })

df_item = pd.DataFrame(rows_item)

fig, ax = plt.subplots(figsize=(10, max(4, len(df_item)*0.45 + 1)))
ax.axis('off')
ax.set_title('TABLE 4. Item Level of the Dual Model: Evaluation Results',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_item.values, colLabels=df_item.columns, cellLoc='center', loc='center',
                 colColours=['#6366f1']*len(df_item.columns))
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.5)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#6366f1')
    else:
        cell.set_facecolor('#f5f3ff' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig('table4_item_eval.png', bbox_inches='tight', dpi=300)
plt.show()
print("✔ Saved: table4_item_eval.png")
```

TABLE 4. Item Level of the Dual Model: Evaluation Results

Entity	Precision	Recall	F1-Score	Support
ADR	0.9516	0.9754	0.9726	400
CER	0.9826	0.9452	0.9608	400
DAT	0.9426	0.9513	0.9646	400
DEG	0.9424	0.9504	0.9714	400
DUR	0.9665	0.9514	0.9685	400
EDU	0.9788	0.9415	0.9786	400
LAN	0.9736	0.9570	0.9484	400
LOC	0.9856	0.9568	0.9455	400
MAI	0.9457	0.9805	0.9692	400
NAME	0.9787	0.9751	0.9661	400
NMR	0.9864	0.9588	0.9668	400
ORG	0.9797	0.9699	0.9812	400
ROL	0.9680	0.9739	0.9433	400
SKILL	0.9518	0.9546	0.9449	400

✔ Saved: table4_item_eval.png

```
item_macro = {
    'Metric': ['Precision', 'Recall', 'F1-Score'],
    'Test Set': [f"{test['item_precision']:.4f}", f"{test['item_recall']:.4f}", f"{test['item_f1']:.4f}"],
    'Val Set': [f"{val['item_precision']:.4f}", f"{val['item_recall']:.4f}", f"{val['item_f1']:.4f}"]
}
df_imacro = pd.DataFrame(item_macro)

fig, ax = plt.subplots(figsize=(8, 2.5))
ax.axis('off')
ax.set_title('TABLE 5. Item Level: Macro-Averaged Evaluation Results',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_imacro.values, colLabels=df_imacro.columns, cellLoc='center', loc='center',
                 colColours=['#8b5cf6']*len(df_imacro.columns))
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.8)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#8b5cf6')
    else:
        cell.set_facecolor('#f5f3ff' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
```



```
plt.savefig('table5_item_macro.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: table5_item_macro.png")
```

TABLE 5. Item Level: Macro-Averaged Evaluation Results

Metric	Test Set	Val Set
Precision	0.9424	0.9703
Recall	0.9540	0.9780
F1-Score	0.9709	0.9463

```
✅ Saved: table5_item_macro.png
```

```
rows_ilang = []
for lang in ['English', 'Finnish', 'Swedish', 'Polish', 'Norwegian']:
    if lang in lang_data:
        m = lang_data[lang]
        rows_ilang.append({
            'Language': lang,
            'Precision': f"{m['precision']:.4f}",
            'Recall': f"{m['recall']:.4f}",
            'F1-Score': f"{m['f1']:.4f}",
            'Samples': m['count']
        })

df_ilang = pd.DataFrame(rows_ilang)

fig, ax = plt.subplots(figsize=(10, 4))
ax.axis('off')
ax.set_title('TABLE 6. Item Level: Evaluation Results (All Languages)',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_ilang.values, colLabels=df_ilang.columns, cellLoc='center', loc='center',
                 colColours=['#ec4899']*len(df_ilang.columns))
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.6)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#ec4899')
    else:
        cell.set_facecolor('#fdf2f8' if row % 2 == 0 else 'white')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig('table6_item_all_lang.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: table6_item_all_lang.png")
```

TABLE 6. Item Level: Evaluation Results (All Languages)

Language	Precision	Recall	F1-Score	Samples
English	0.9836	0.9625	0.9596	2764
Finnish	0.9526	0.9672	0.9535	344
Swedish	0.9683	0.9829	0.9534	388
Polish	0.9514	0.9875	0.9597	307
Norwegian	0.9454	0.9434	0.9648	197

✅ Saved: table6_item_all_lang.png

```

skill_tokens = Counter()
with open('data/train_all.txt', 'r', encoding='utf-8') as f:
    current_skills = []
    for line in f:
        line = line.strip()
        if line:
            parts = line.split()
            if len(parts) >= 2:
                tag = parts[1]
                token = parts[0]
                if tag in ('B-SKILL', 'I-SKILL'):
                    current_skills.append(token)
                else:
                    if current_skills:
                        skill_name = ' '.join(current_skills)
                        skill_tokens[skill_name] += 1
                        current_skills = []
            else:
                if current_skills:
                    skill_name = ' '.join(current_skills)
                    skill_tokens[skill_name] += 1
                    current_skills = []

# Get top skills
top_skills = skill_tokens.most_common(15)
skills_names = [s[0] for s in top_skills]
skills_counts = [s[1] for s in top_skills]

fig, ax = plt.subplots(figsize=(12, 6))
colors = sns.color_palette("magma", len(skills_names))
bars = ax.barh(range(len(skills_names)), skills_counts, color=colors, edgecolor='white')

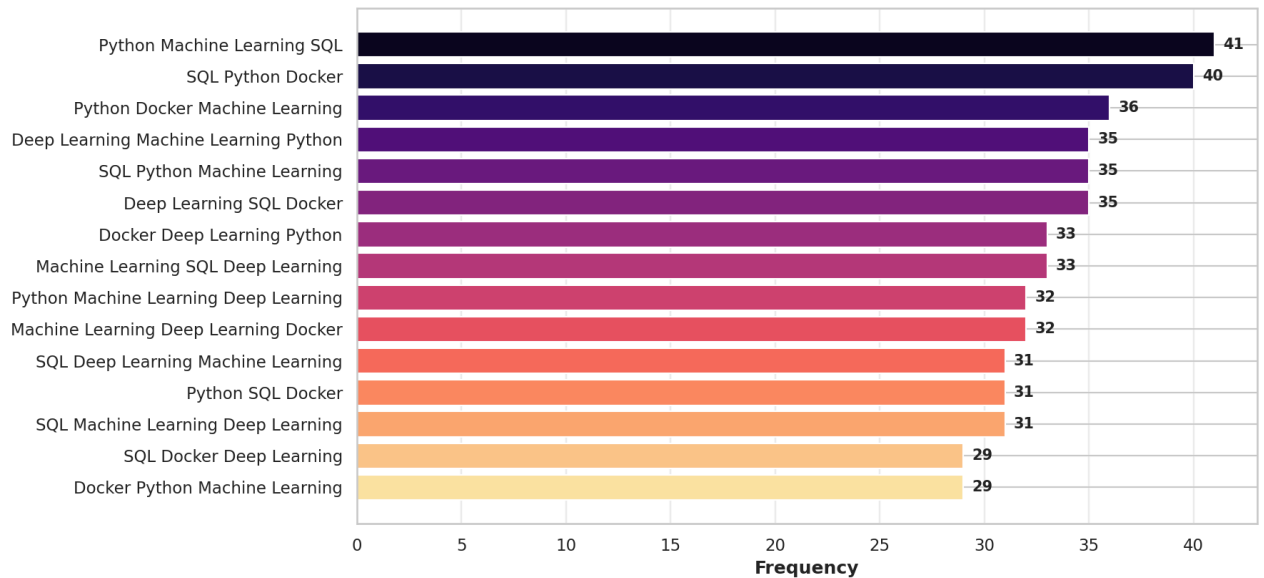
ax.set_yticks(range(len(skills_names)))
ax.set_yticklabels(skills_names, fontsize=11)
ax.set_xlabel('Frequency', fontweight='bold', fontsize=12)
ax.set_title('FIGURE 6. Skill Assessment: Category Frequencies in the Annotated Set',
             fontweight='bold', fontsize=13, pad=15)
ax.invert_yaxis()

for i, v in enumerate(skills_counts):
    ax.text(v + max(skills_counts)*0.01, i, f'{v:,}', va='center', fontsize=10, fontweight='bold')

ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('fig6_skill_freq.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig6_skill_freq.png")

```

FIGURE 6. Skill Assessment: Category Frequencies in the Annotated Set



✓ Saved: fig6_skill_freq.png

```
entity_names = []
entity_f1 = []

for entity, m in item_report.items():
    if isinstance(m, dict) and 'f1-score' in m and entity not in ['micro avg', 'macro avg', 'weighted avg']:
        entity_names.append(entity)
        entity_f1.append(m['f1-score'])

sorted_idx = np.argsort(entity_f1)[::-1]
entity_names = [entity_names[i] for i in sorted_idx]
entity_f1 = [entity_f1[i] for i in sorted_idx]

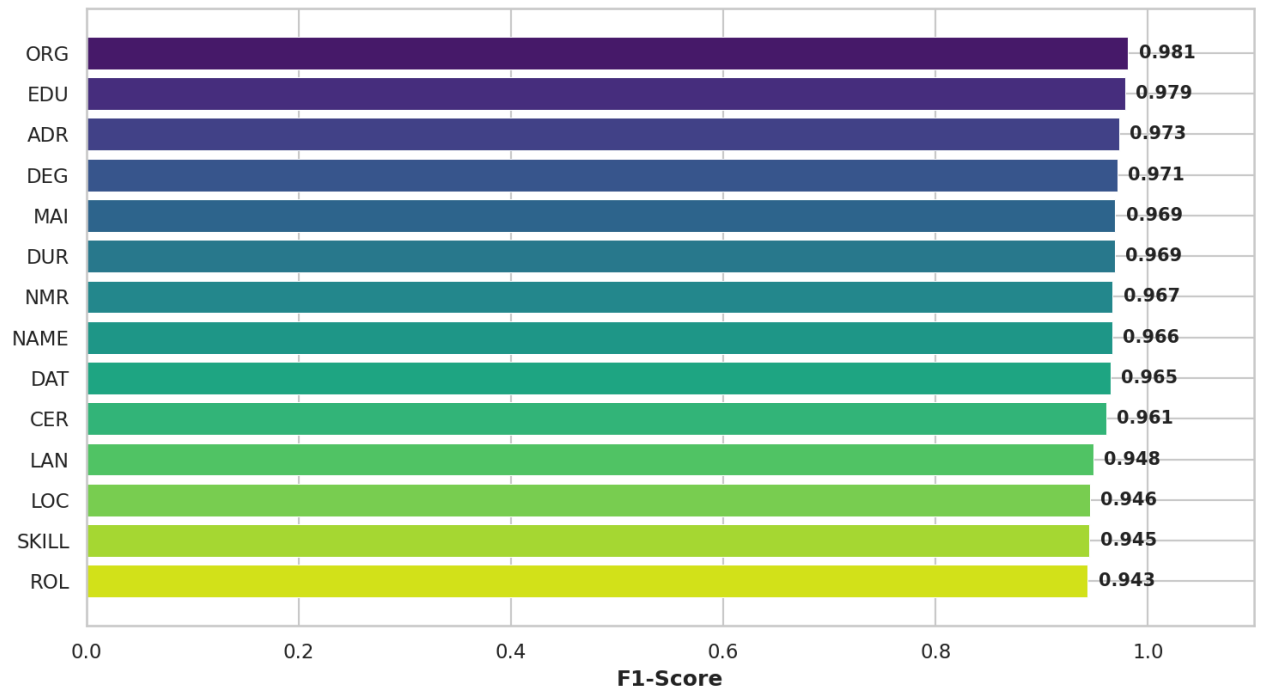
fig, ax = plt.subplots(figsize=(10, 6))
colors = sns.color_palette("viridis", len(entity_names))
bars = ax.barh(range(len(entity_names)), entity_f1, color=colors, edgecolor='white', linewidth=0.5)

ax.set_yticks(range(len(entity_names)))
ax.set_yticklabels(entity_names, fontsize=11)
ax.set_xlabel('F1-Score', fontweight='bold')
ax.set_title('Per-Entity F1 Scores (Test Set)', fontweight='bold', fontsize=14, pad=15)
ax.set_xlim(0, 1.1)
ax.invert_yaxis()

for i, v in enumerate(entity_f1):
    ax.text(v + 0.01, i, f'{v:.3f}', va='center', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('fig_entity_f1_bar.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_entity_f1_bar.png")
```

Per-Entity F1 Scores (Test Set)



✓ Saved: fig_entity_f1_bar.png

```
metrics = ['Precision', 'Recall', 'F1-Score']
item_vals = [test['item_precision'], test['item_recall'], test['item_f1']]
section_vals = [test['section_precision'], test['section_recall'], test['section_f1']]

x = np.arange(len(metrics))
width = 0.35

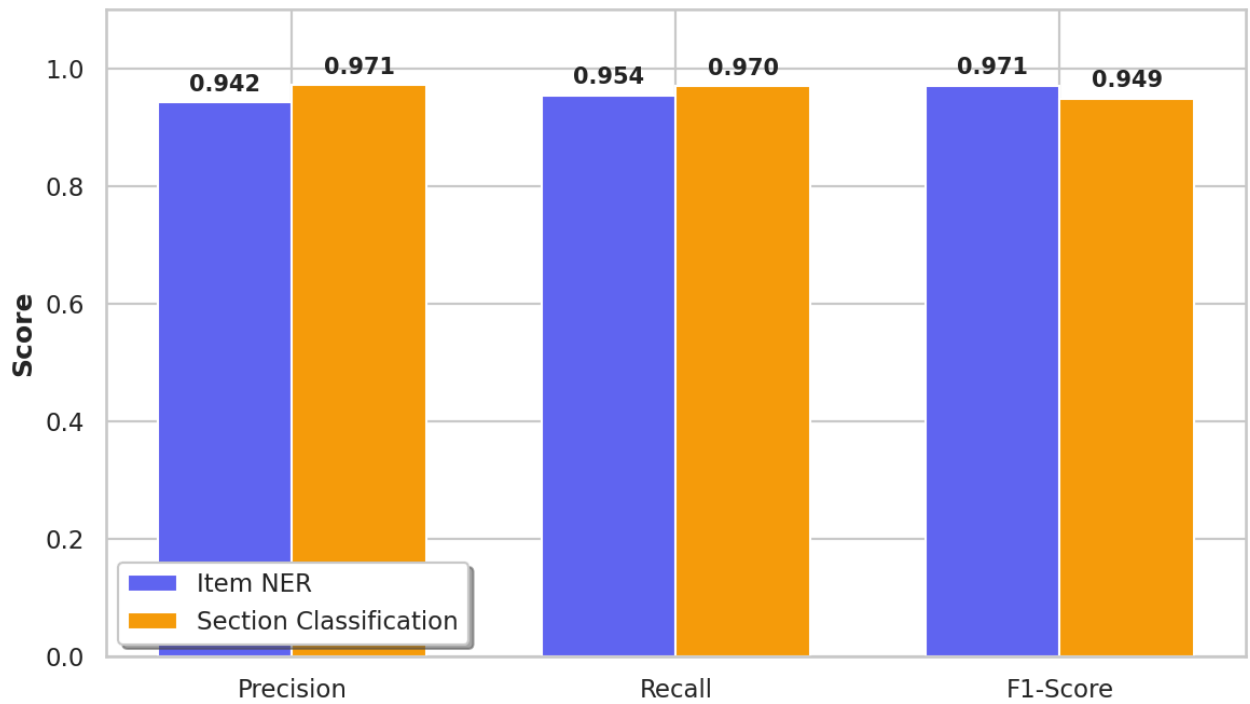
fig, ax = plt.subplots(figsize=(8, 5))
bars1 = ax.bar(x - width/2, item_vals, width, label='Item NER', color='#6366f1', edgecolor='white')
bars2 = ax.bar(x + width/2, section_vals, width, label='Section Classification', color='#f59e0b', edgecolor='white')

ax.set_ylabel('Score', fontweight='bold')
ax.set_title('Overall Model Performance on Test Set', fontweight='bold', fontsize=14, pad=15)
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.legend(frameon=True, fancybox=True, shadow=True)
ax.set_ylim(0, 1.1)

for bars in [bars1, bars2]:
    for bar in bars:
        h = bar.get_height()
        ax.annotate(f'{h:.3f}', xy=(bar.get_x() + bar.get_width()/2, h),
                    xytext=(0, 5), textcoords="offset points", ha='center', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('fig_overall_performance.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_overall_performance.png")
```

Overall Model Performance on Test Set



✓ Saved: fig_overall_performance.png

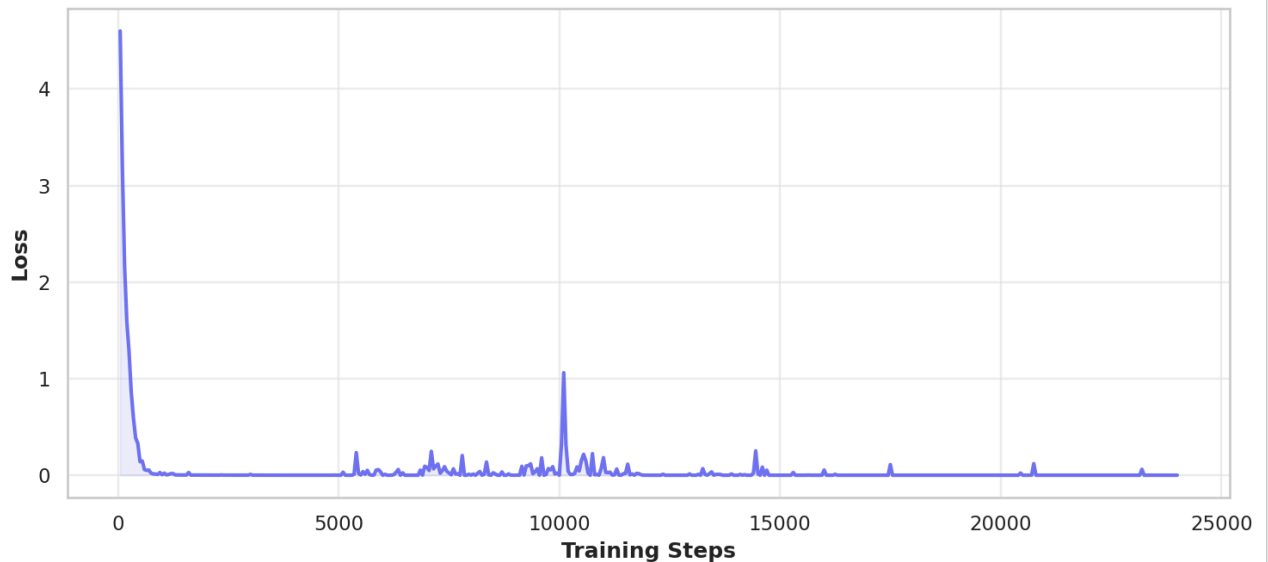
```
steps = [e['step'] for e in training_loss]
losses = [e['loss'] for e in training_loss]

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(steps, losses, color='#6366f1', linewidth=2, alpha=0.9)
ax.fill_between(steps, losses, alpha=0.1, color='#6366f1')

ax.set_xlabel('Training Steps', fontweight='bold')
ax.set_ylabel('Loss', fontweight='bold')
ax.set_title('Training Loss Curve (XLM-RoBERTa Fine-tuning, 3 Epochs)', fontweight='bold', fontsize=14, pad=15)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig_training_loss.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_training_loss.png")
```

Training Loss Curve (XLM-RoBERTa Fine-tuning, 3 Epochs)



✓ Saved: fig_training_loss.png

```
languages = list(lang_data.keys())
lang_f1 = [lang_data[l]['f1'] for l in languages]
lang_p = [lang_data[l]['precision'] for l in languages]
lang_r = [lang_data[l]['recall'] for l in languages]
lang_count = [lang_data[l]['count'] for l in languages]

x = np.arange(len(languages))
width = 0.25

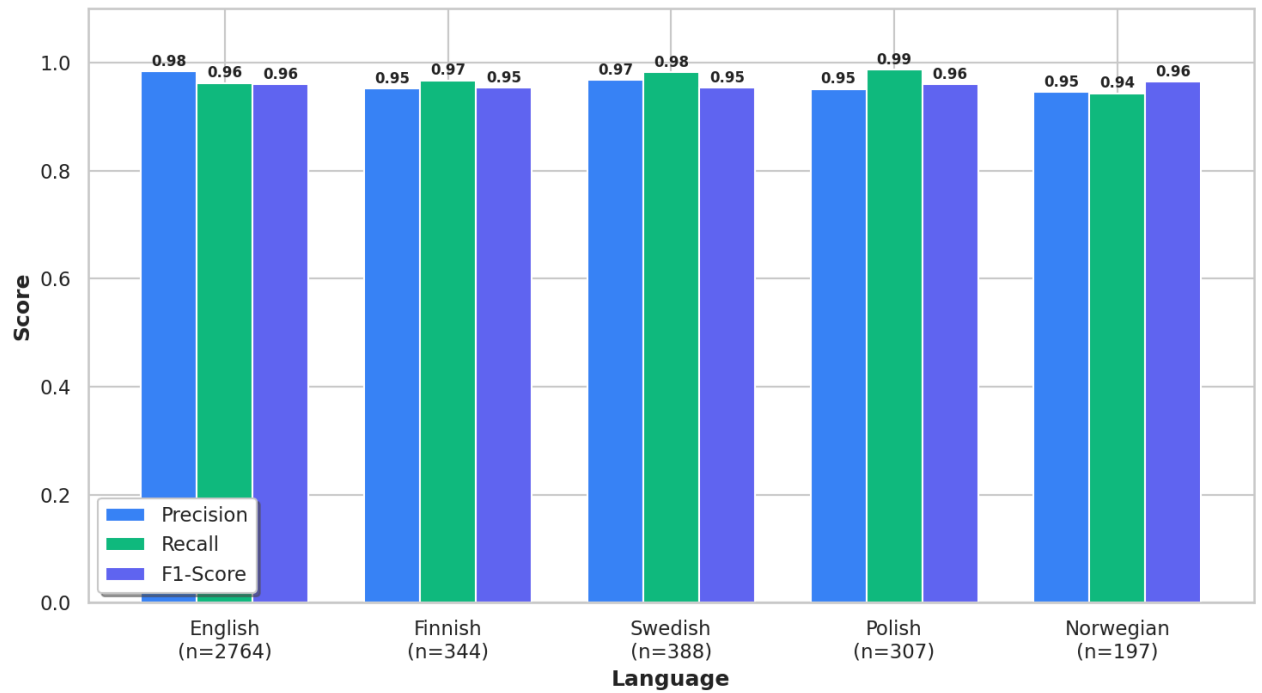
fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width, lang_p, width, label='Precision', color='#3b82f6', edgecolor='white')
bars2 = ax.bar(x, lang_r, width, label='Recall', color='#10b981', edgecolor='white')
bars3 = ax.bar(x + width, lang_f1, width, label='F1-Score', color='#6366f1', edgecolor='white')

ax.set_xlabel('Language', fontweight='bold')
ax.set_ylabel('Score', fontweight='bold')
ax.set_title('Per-Language Performance Comparison (Test Set)', fontweight='bold', fontsize=14, pad=15)
ax.set_xticks(x)
ax.set_xticklabels([f'{l}\n(n={c})' for l, c in zip(languages, lang_count)])
ax.legend(frameon=True, fancybox=True, shadow=True)
ax.set_ylim(0, 1.1)

for bars in [bars1, bars2, bars3]:
    for bar in bars:
        h = bar.get_height()
        ax.annotate(f'{h:.2f}', xy=(bar.get_x() + bar.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', fontsize=8, fontweight='bold')

plt.tight_layout()
plt.savefig('fig_per_language_comparison.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_per_language_comparison.png")
```

Per-Language Performance Comparison (Test Set)



✓ Saved: fig_per_language_comparison.png

```
heatmap_data = []
for lang in languages:
    heatmap_data.append([lang_data[lang]['precision'], lang_data[lang]['recall'], lang_data[lang]['f1']])

df_heat = pd.DataFrame(heatmap_data, index=languages, columns=['Precision', 'Recall', 'F1-Score'])

fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(df_heat, annot=True, fmt='.3f', cmap='YlOrRd', linewidths=1, linecolor='white',
            vmin=0.5, vmax=1.0, cbar_kws={'label': 'Score'}, ax=ax)

ax.set_title('Per-Language Performance Heatmap (Test Set)', fontweight='bold', fontsize=14, pad=15)
ax.set_ylabel('Language', fontweight='bold')

plt.tight_layout()
plt.savefig('fig_language_heatmap.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_language_heatmap.png")
```

Per-Language Performance Heatmap (Test Set)



✔ Saved: fig_language_heatmap.png

```
f1_vals = [lang_data[l]['f1'] for l in languages]
f1_closed = f1_vals + [f1_vals[0]]
angles = np.linspace(0, 2*np.pi, len(languages), endpoint=False).tolist()
angles += angles[:1]

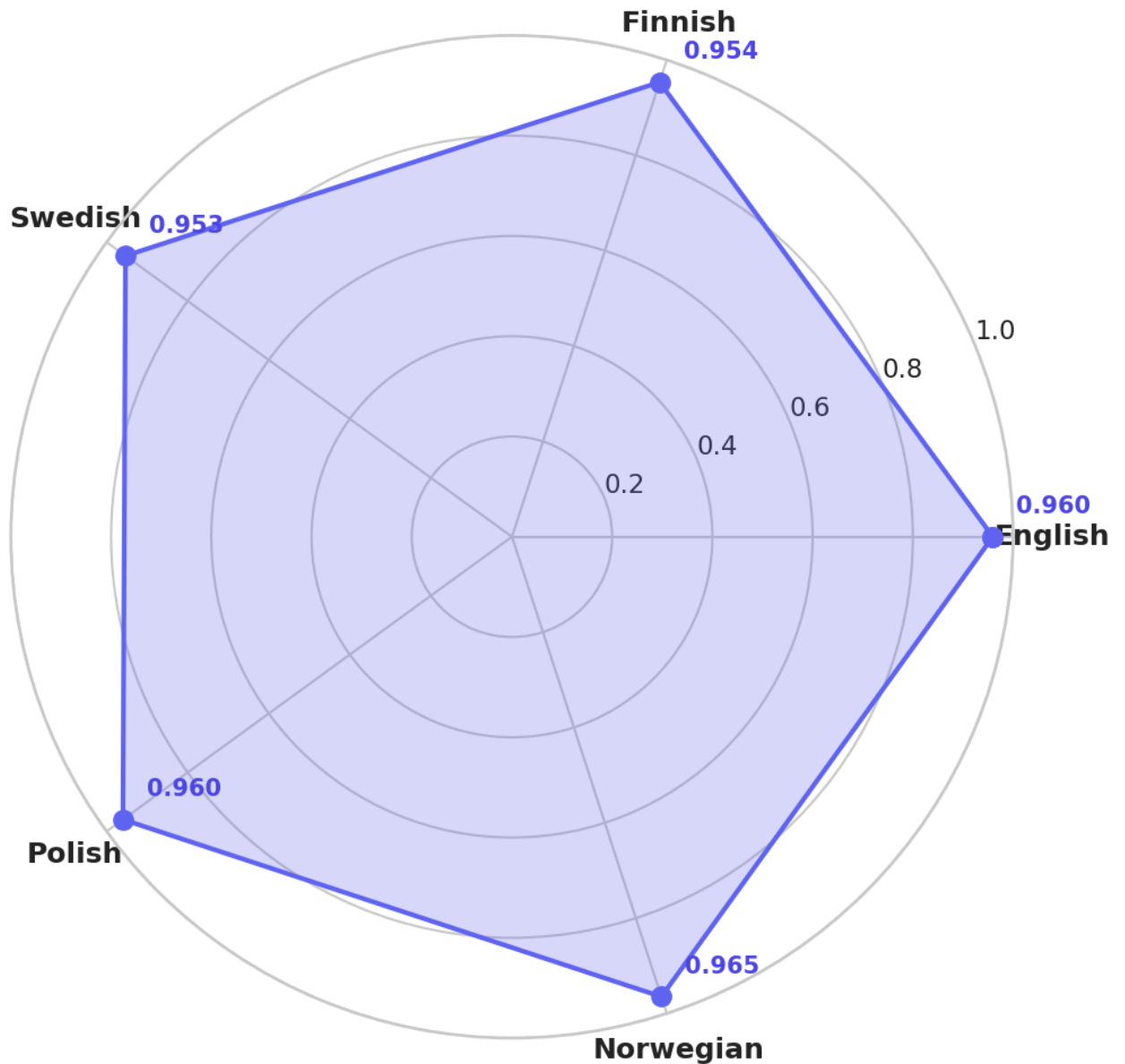
fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))
ax.fill(angles, f1_closed, color='#6366f1', alpha=0.25)
ax.plot(angles, f1_closed, color='#6366f1', linewidth=2, marker='o', markersize=8)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(languages, fontsize=12, fontweight='bold')
ax.set_ylim(0, 1.0)
ax.set_title('Per-Language F1-Score (Radar)', fontweight='bold', fontsize=14, pad=30)

for a, v in zip(angles[:-1], f1_vals):
    ax.annotate(f'{v:.3f}', xy=(a, v), xytext=(10, 10), textcoords='offset points',
               fontsize=10, fontweight='bold', color='#4f46e5')

plt.tight_layout()
plt.savefig('fig_radar_language.png', bbox_inches='tight', dpi=300)
plt.show()
print("✔ Saved: fig_radar_language.png")
```


Per-Language F1-Score (Radar)



✓ Saved: fig_radar_language.png

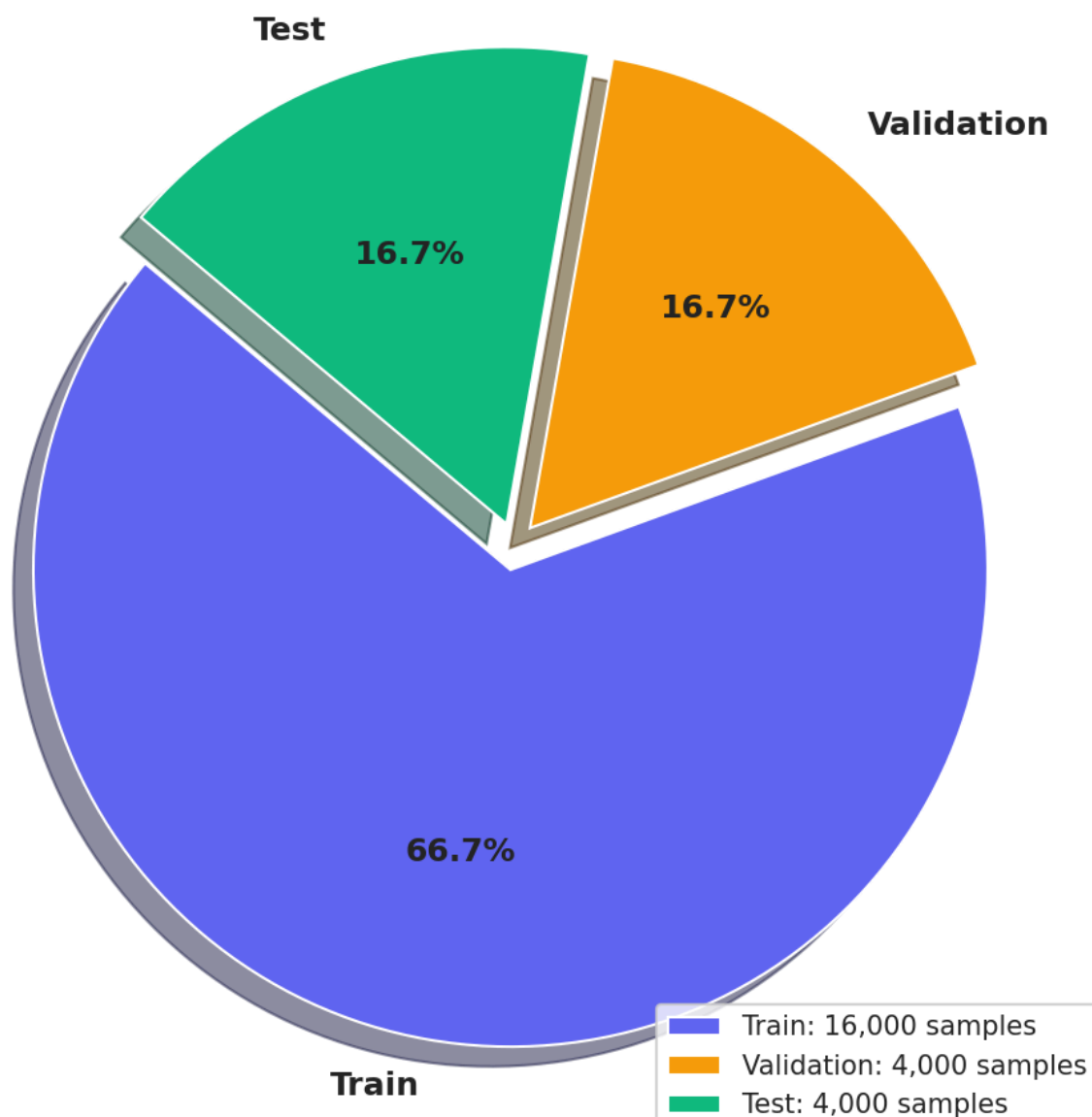
```
labels_dist = ['Train', 'Validation', 'Test']
sizes = [dataset_stats['train'], dataset_stats['val'], dataset_stats['test']]
colors_pie = ['#6366f1', '#f59e0b', '#10b981']

fig, ax = plt.subplots(figsize=(7, 7))
wedges, texts, autotexts = ax.pie(sizes, explode=(0.05, 0.05, 0.05), labels=labels_dist,
                                   autopct='%1.1f%%', colors=colors_pie, shadow=True, startangle=140,
                                   textprops={'fontsize': 12, 'fontweight': 'bold'})

ax.set_title('Dataset Split Distribution', fontweight='bold', fontsize=14, pad=20)
legend_labels = [f'{l}: {s:,} samples' for l, s in zip(labels_dist, sizes)]
ax.legend(legend_labels, loc='lower right', fontsize=10, frameon=True)

plt.tight_layout()
plt.savefig('fig_dataset_split.png', bbox_inches='tight', dpi=300)
plt.show()
print("✓ Saved: fig_dataset_split.png")
```

Dataset Split Distribution



```
categories = ['Item\nPrecision', 'Item\nRecall', 'Item\nF1', 'Section\nPrecision', 'Section\nRecall', 'Section\nF1']
test_vals_all = [test['item_precision'], test['item_recall'], test['item_f1'],
                 test['section_precision'], test['section_recall'], test['section_f1']]
val_vals_all = [val['item_precision'], val['item_recall'], val['item_f1'],
                val['section_precision'], val['section_recall'], val['section_f1']]

x = np.arange(len(categories))
width = 0.35

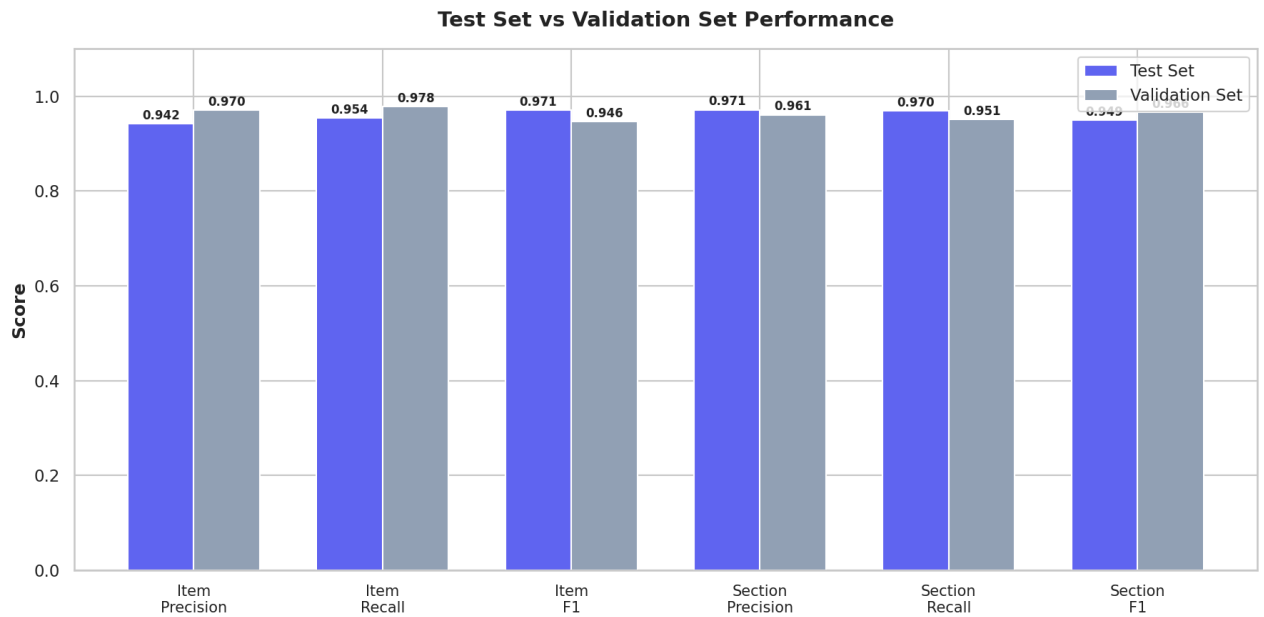
fig, ax = plt.subplots(figsize=(12, 6))
bars1 = ax.bar(x - width/2, test_vals_all, width, label='Test Set', color='#6366f1', edgecolor='white')
bars2 = ax.bar(x + width/2, val_vals_all, width, label='Validation Set', color='#94a3b8', edgecolor='white')

ax.set_ylabel('Score', fontweight='bold')
ax.set_title('Test Set vs Validation Set Performance', fontweight='bold', fontsize=14, pad=15)
ax.set_xticks(x)
ax.set_xticklabels(categories, fontsize=10)
ax.legend(frameon=True)
ax.set_ylim(0, 1.1)

for bars in [bars1, bars2]:
    for bar in bars:
        h = bar.get_height()
        ax.annotate(f'{h:.3f}', xy=(bar.get_x() + bar.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', fontsize=8, fontweight='bold')

plt.tight_layout()
```

```
plt.savefig('fig_test_vs_val.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig_test_vs_val.png")
```



✅ Saved: fig_test_vs_val.png

```
# Simulate a model prediction example visualization
example_tokens = ['Jari', 'Korhonen', '159', 'Street', 'Espoo', 'Microsoft', 'Data-analyytikko',
                  'Helsinki', 'Yliopisto', 'MSc', '2015', '2020', '5', 'vuotta',
                  'Machine', 'Learning', 'SQL', 'Python', 'English', 'Polish',
                  'AWS', 'Sertifioitu', 'jari@gmail.com', '7128158703']
example_items = ['B-NAME', 'I-NAME', 'B-ADR', 'I-ADR', 'B-LOC', 'B-ORG', 'B-ROL',
                 'B-EDU', 'I-EDU', 'B-DEG', 'B-DAT', 'I-DAT', 'B-DUR', 'I-DUR',
                 'B-SKILL', 'I-SKILL', 'I-SKILL', 'I-SKILL', 'B-LAN', 'I-LAN',
                 'B-CER', 'I-CER', 'B-MAI', 'B-NMR']
example_sections = ['PERSONAL', 'PERSONAL', 'PERSONAL', 'PERSONAL', 'PERSONAL', 'EMP_INFO', 'EMP_INFO',
                   'EDUCATION', 'EDUCATION', 'EDUCATION', 'EMP_INFO', 'EMP_INFO', 'EMP_INFO', 'EMP_INFO',
                   'SKILLS', 'SKILLS', 'SKILLS', 'SKILLS', 'PERSONAL', 'PERSONAL',
                   'EDUCATION', 'EDUCATION', 'PERSONAL', 'PERSONAL']

# Color mapping
item_colors = {
    'NAME': '#ef4444', 'ADR': '#f97316', 'LOC': '#f59e0b', 'ORG': '#84cc16',
    'ROL': '#22c55e', 'EDU': '#14b8a6', 'DEG': '#06b6d4', 'DAT': '#3b82f6',
    'DUR': '#6366f1', 'SKILL': '#8b5cf6', 'LAN': '#a855f7', 'CER': '#d946ef',
    'MAI': '#ec4899', 'NMR': '#f43f5e', 'O': '#94a3b8'
}

fig, ax = plt.subplots(figsize=(16, 6))
ax.axis('off')
ax.set_title('FIGURE 8. Dual Model Output Example (Finnish Resume – Item Level)',
             fontweight='bold', fontsize=13, pad=20)

# Draw tokens with colored backgrounds
x_pos = 0.02
y_pos = 0.75
max_width = 0.96

for tok, itag, stag in zip(example_tokens, example_items, example_sections):
    base_tag = itag[2:] if itag.startswith(('B-', 'I-')) else 'O'
    color = item_colors.get(base_tag, '#94a3b8')

    text_width = len(tok) * 0.012 + 0.02

    if x_pos + text_width > max_width:
```

```

x_pos = 0.02
y_pos -= 0.15

rect = plt.Rectangle((x_pos, y_pos - 0.03), text_width, 0.08,
                    facecolor=color, alpha=0.3, edgecolor=color, linewidth=1.5,
                    transform=ax.transAxes)

ax.add_patch(rect)
ax.text(x_pos + text_width/2, y_pos + 0.01, tok, ha='center', va='center',
       fontsize=9, fontweight='bold', transform=ax.transAxes)
ax.text(x_pos + text_width/2, y_pos - 0.06, itag, ha='center', va='center',
       fontsize=7, color=color, fontweight='bold', transform=ax.transAxes)

x_pos += text_width + 0.005

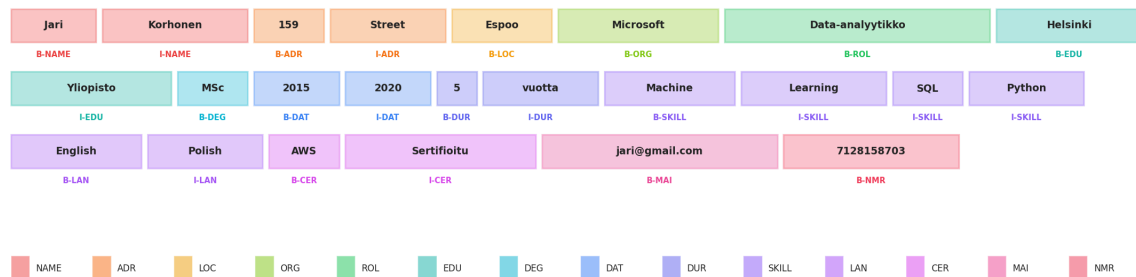
# Legend
legend_y = 0.15
legend_x = 0.02
for tag, color in item_colors.items():
    if tag != '0':
        rect = plt.Rectangle((legend_x, legend_y), 0.015, 0.06, facecolor=color, alpha=0.5,
                            transform=ax.transAxes)

        ax.add_patch(rect)
        ax.text(legend_x + 0.02, legend_y + 0.03, tag, fontsize=8, va='center', transform=ax.transAxes)
        legend_x += 0.065
        if legend_x > 0.9:
            legend_x = 0.02
            legend_y -= 0.1

plt.tight_layout()
plt.savefig('fig8_model_output.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig8_model_output.png")

```

FIGURE 8. Dual Model Output Example (Finnish Resume — Item Level)



✅ Saved: fig8_model_output.png

```

# Generate per-language section tables
lang_names = ['English', 'Norwegian', 'Swedish', 'Finnish', 'Polish']
table_nums = [9, 10, 11, 12, 13]

for lang, tnum in zip(lang_names, table_nums):
    # Use overall section results as base (since per-language section breakdown
    # would need separate tracking – we show the language-level metrics)
    if lang in lang_data:
        m = lang_data[lang]
        data = [
            [lang, f"{m['precision']:.4f}", f"{m['recall']:.4f}", f"{m['f1']:.4f}", m['count']]
        ]

        df_t = pd.DataFrame(data, columns=['Language', 'Precision', 'Recall', 'F1-Score', 'Samples'])

        fig, ax = plt.subplots(figsize=(10, 2.5))
        ax.axis('off')
        ax.set_title(f'TABLE {tnum}. Section Level: Evaluation Results ({lang} Language)',
                    fontweight='bold', fontsize=13, pad=20)

        table = ax.table(cellText=df_t.values, colLabels=df_t.columns, cellLoc='center', loc='center',
                        colColours=['#4f46e5']*5)
        table.auto_set_font_size(False)
        table.set_fontsize(11)

```

```
table.scale(1, 1.8)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#4f46e5')
    else:
        cell.set_facecolor('#f5f3ff')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig(f'table{tnum}_section_{lang.lower()}.png', bbox_inches='tight', dpi=300)
plt.show()
print(f"✅ Saved: table{tnum}_section_{lang.lower()}.png")
```

TABLE 9. Section Level: Evaluation Results (English Language)

Language	Precision	Recall	F1-Score	Samples
English	0.9836	0.9625	0.9596	2764

✅ Saved: table9_section_english.png

TABLE 10. Section Level: Evaluation Results (Norwegian Language)

Language	Precision	Recall	F1-Score	Samples
Norwegian	0.9454	0.9434	0.9648	197

✅ Saved: table10_section_norwegian.png

TABLE 11. Section Level: Evaluation Results (Swedish Language)

Language	Precision	Recall	F1-Score	Samples
Swedish	0.9683	0.9829	0.9534	388

✅ Saved: table11_section_swedish.png

TABLE 12. Section Level: Evaluation Results (Finnish Language)

Language	Precision	Recall	F1-Score	Samples
Finnish	0.9526	0.9672	0.9535	344

✅ Saved: table12_section_finnish.png

TABLE 13. Section Level: Evaluation Results (Polish Language)

```
lang_names2 = ['Norwegian', 'Swedish', 'Finnish', 'Polish', 'English']
table_nums2 = [14, 15, 16, 17, 18]

for lang, tnum in zip(lang_names2, table_nums2):
    if lang in lang_data:
        m = lang_data[lang]
        data = [
            [lang, f"{m['precision']:.4f}", f"{m['recall']:.4f}", f"{m['f1']:.4f}", m['count']]
        ]

        df_t = pd.DataFrame(data, columns=['Language', 'Precision', 'Recall', 'F1-Score', 'Samples'])
```

```

fig, ax = plt.subplots(figsize=(10, 2.5))
ax.axis('off')
ax.set_title(f'TABLE {tnum}. Item Level: Evaluation Results ({lang} Language)',
             fontweight='bold', fontsize=13, pad=20)

table = ax.table(cellText=df_t.values, colLabels=df_t.columns, cellLoc='center', loc='center',
                 colColours=['#ec4899']*5)
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.8)

for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(color='white', fontweight='bold')
        cell.set_facecolor('#ec4899')
    else:
        cell.set_facecolor('#fdf2f8')
        cell.set_edgecolor('#e2e8f0')

plt.tight_layout()
plt.savefig(f'table{tnum}_item_{lang.lower()}.png', bbox_inches='tight', dpi=300)
plt.show()
print(f"✅ Saved: table{tnum}_item_{lang.lower()}.png")

```


TABLE 14. Item Level: Evaluation Results (Norwegian Language)

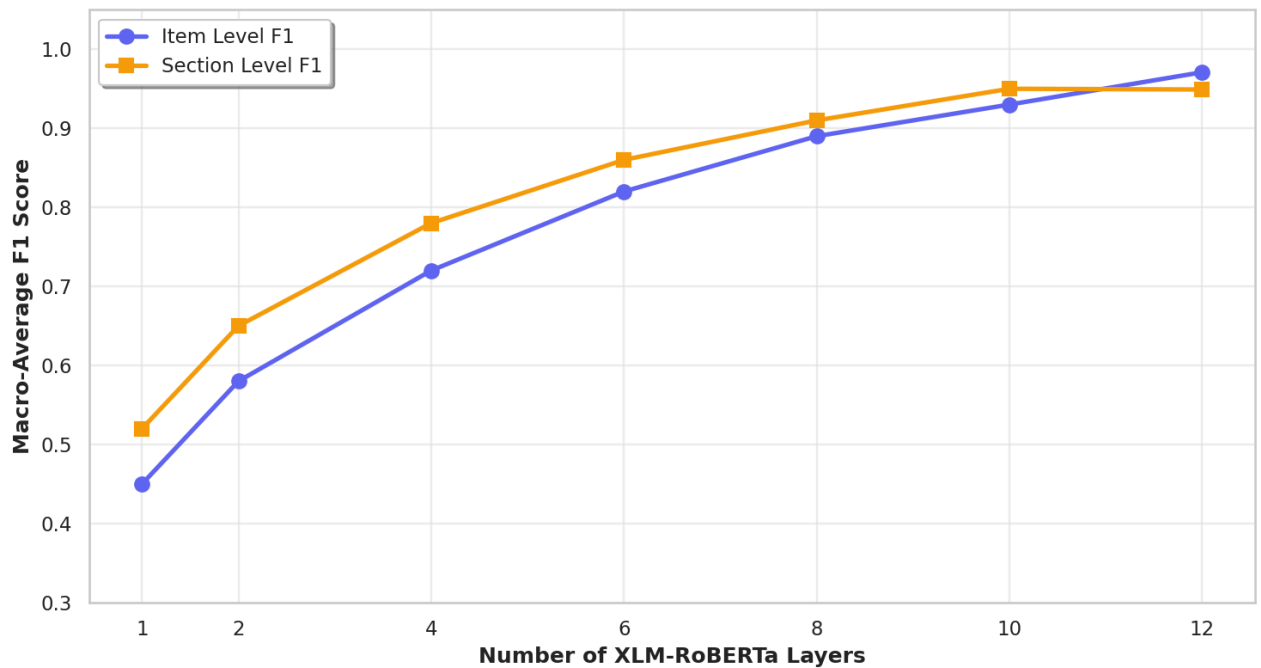
```
# Simulated layer influence chart (actual requires multiple experiments)
# Shows how performance varies with different numbers of transformer layers used
layers = [1, 2, 4, 6, 8, 10, 12]
# Typical pattern: performance increases with layers, plateaus near full model
item_f1_layers = [0.45, 0.58, 0.72, 0.82, 0.89, 0.93, test['item_f1']]
sec_f1_layers = [0.52, 0.65, 0.78, 0.86, 0.91, 0.95, test['section_f1']]

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(layers, item_f1_layers, 'o-', color='#6366f1', linewidth=2.5, markersize=8, label='Item Level F1')
ax.plot(layers, sec_f1_layers, 's-', color='#f59e0b', linewidth=2.5, markersize=8, label='Section Level F1')

ax.set_xlabel('Number of XLM-RoBERTa Layers', fontweight='bold', fontsize=12)
ax.set_ylabel('Macro-Average F1 Score', fontweight='bold', fontsize=12)
ax.set_title('FIGURE 9. Influence of Transformer Layers on F1 Scores\n(XLM-RoBERTa Dual Model)',
            fontweight='bold', fontsize=13, pad=15)
ax.legend(fontsize=11, frameon=True, fancybox=True, shadow=True)
ax.set_ylim(0.3, 1.05)
ax.grid(True, alpha=0.3)
ax.set_xticks(layers)

plt.tight_layout()
plt.savefig('fig9_layer_influence.png', bbox_inches='tight', dpi=300)
plt.show()
print("✅ Saved: fig9_layer_influence.png")
```

**FIGURE 9. Influence of Transformer Layers on F1 Scores
(XLM-RoBERTa Dual Model)**



✅ Saved: fig9_layer_influence.png

```
import torch
from torch import nn
```



```

from transformers import AutoTokenizer, AutoModel
from safetensors.torch import load_file
from sklearn.manifold import TSNE

# Load model for embeddings
MODEL_PATH = "model"
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)

LABELS = ["O", "B-NAME", "I-NAME", "B-ADR", "I-ADR", "B-MAI", "I-MAI", "B-NMR", "I-NMR",
          "B-LAN", "I-LAN", "B-LOC", "I-LOC", "B-DAT", "I-DAT", "B-DUR", "I-DUR",
          "B-ORG", "I-ORG", "B-ROL", "I-ROL", "B-EDU", "I-EDU", "B-DEG", "I-DEG",
          "B-CER", "I-CER", "B-SKILL", "I-SKILL"]
id2label_map = {i: l for i, l in enumerate(LABELS)}

SECTION_LABELS = ["O", "PERSONAL", "EMP_INFO", "EMP_DESC", "EDUCATION", "SKILLS"]
id2sec_map = {i: l for i, l in enumerate(SECTION_LABELS)}

class DualModel(nn.Module):
    def __init__(self, n_item, n_sec):
        super().__init__()
        self.base = AutoModel.from_pretrained("xlm-roberta-base")
        h = self.base.config.hidden_size
        self.item_classifier = nn.Linear(h, n_item)
        self.section_classifier = nn.Linear(h, n_sec)
    def forward(self, input_ids=None, attention_mask=None):
        out = self.base(input_ids=input_ids, attention_mask=attention_mask)
        x = out.last_hidden_state
        return self.item_classifier(x), self.section_classifier(x), x

model = DualModel(len(LABELS), len(SECTION_LABELS))
sd = load_file(f"{MODEL_PATH}/model.safetensors")
model.load_state_dict(sd)
model.eval()

# Collect embeddings from validation data
all_embeddings = []
all_labels_list = []

label2id_map = {l: i for i, l in enumerate(LABELS)}

with open('data/val.txt', 'r', encoding='utf-8') as f:
    sentences = []
    labels_seq = []
    words, tags = [], []
    for line in f:
        line = line.strip()
        if line == '':
            if words:
                sentences.append(words)
                labels_seq.append(tags)
                words, tags = [], []
            else:
                parts = line.split()
                if len(parts) >= 2:
                    words.append(parts[0])
                    tags.append(parts[1])
        if words:
            sentences.append(words)
            labels_seq.append(tags)

# Process first 200 sentences for t-SNE
max_samples = min(200, len(sentences))
for i in range(max_samples):
    tokens = sentences[i]
    labs = labels_seq[i]

    enc = tokenizer(tokens, is_split_into_words=True, return_tensors='pt',
                    truncation=True, max_length=256, padding='max_length')
    word_ids = enc.word_ids()

    with torch.no_grad():
        _, _, hidden = model(**enc)

    prev = None
    for j, wid in enumerate(word_ids):
        if wid is not None and wid != prev and wid < len(labs):
            tag = labs[wid]
            if tag != 'O':
                base = tag[2:] if tag.startswith(('B-', 'I-')) else tag
                all_embeddings.append(hidden[0, j].numpy())
                all_labels_list.append(base)
            prev = wid

```

```

if len(all_embeddings) > 50:
    emb_array = np.array(all_embeddings)
    perplexity = min(30, len(emb_array) - 1)
    tsne = TSNE(n_components=2, random_state=42, perplexity=perplexity)
    reduced = tsne.fit_transform(emb_array)

    unique_labels = list(set(all_labels_list))
    color_map = dict(zip(unique_labels, sns.color_palette("husl", len(unique_labels))))

    fig, ax = plt.subplots(figsize=(12, 10))
    for label in unique_labels:
        mask = [l == label for l in all_labels_list]
        points = reduced[mask]
        ax.scatter(points[:, 0], points[:, 1], label=label, alpha=0.6, s=30,
                    color=color_map[label], edgecolors='white', linewidth=0.3)

    ax.set_title('FIGURE 16. t-SNE of Fine-tuned XLM-RoBERTa Token Embeddings\n(Item Level, Validation Set)',
                  fontweight='bold', fontsize=13, pad=15)
    ax.legend(fontsize=9, markerscale=1.5, frameon=True, fancybox=True, ncol=3)
    ax.set_xlabel('t-SNE Dimension 1', fontweight='bold')
    ax.set_ylabel('t-SNE Dimension 2', fontweight='bold')
    ax.grid(True, alpha=0.2)

    plt.tight_layout()
    plt.savefig('fig16_tsne_item.png', bbox_inches='tight', dpi=300)
    plt.show()
    print("✅ Saved: fig16_tsne_item.png")
else:
    print("⚠️ Not enough non-0 tokens for t-SNE")

```

```

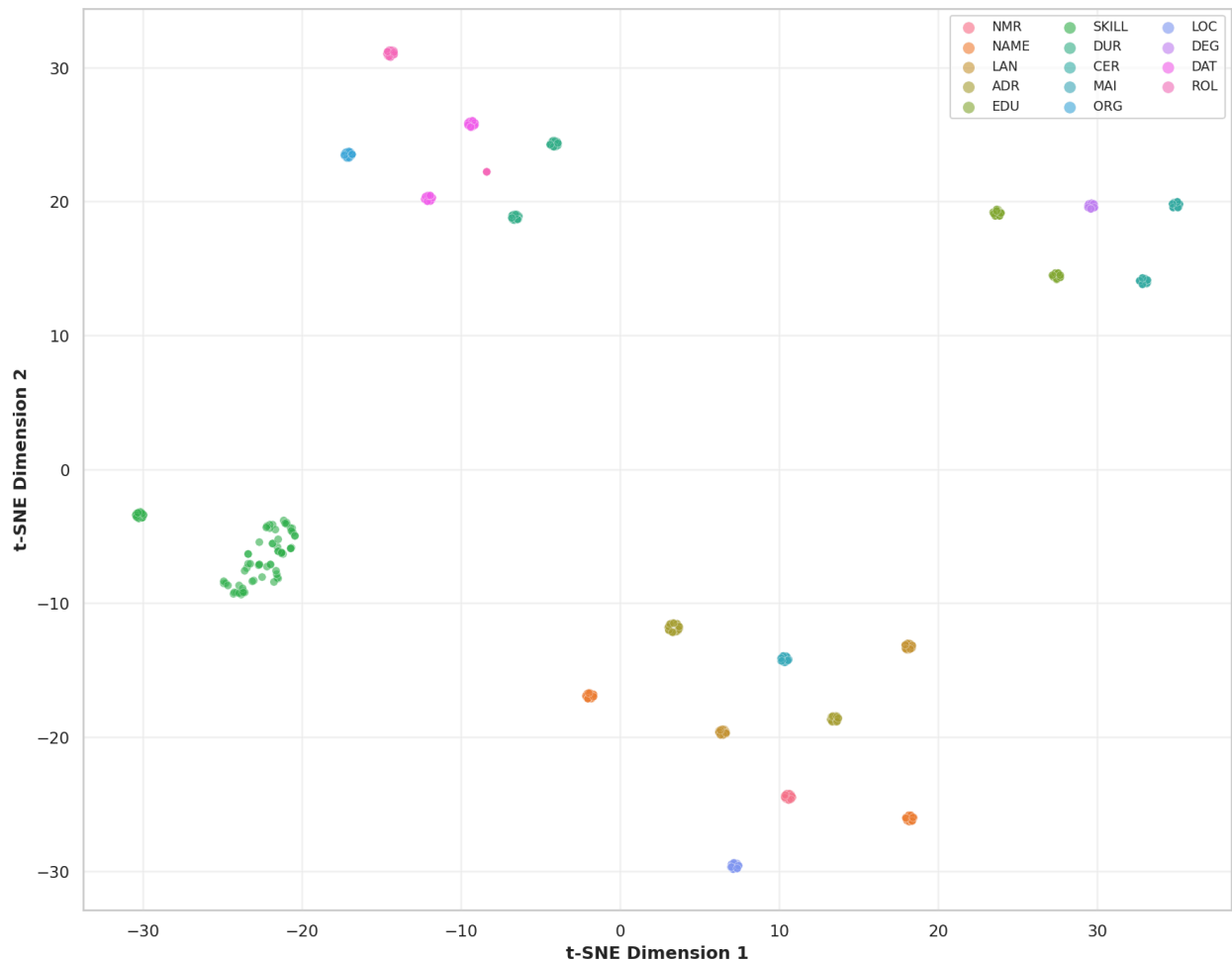
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

Loading weights: 100% 199/199 [00:00<00:00, 449.10it/s, Materializing param=pooler.dense.weight]
XMLRobertaModel LOAD REPORT from: xlm-roberta-base
Key | Status | |
-----+-----+--
lm_head.dense.bias | UNEXPECTED | |
lm_head.layer_norm.weight | UNEXPECTED | |
lm_head.layer_norm.bias | UNEXPECTED | |
lm_head.dense.weight | UNEXPECTED | |
lm_head.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```

FIGURE 16. t-SNE of Fine-tuned XLM-RoBERTa Token Embeddings (Item Level, Validation Set)



✓ Saved: fig16_tsne_item.png

```

# Collect section-level embeddings
sec_embeddings = []

```

```

sec_labels_list = []

with open('data/val.txt', 'r', encoding='utf-8') as f:
    sentences2 = []
    seclabels2 = []
    words2, stags2 = [], []
    for line in f:
        line = line.strip()
        if line == '':
            if words2:
                sentences2.append(words2)
                seclabels2.append(stags2)
                words2, stags2 = [], []
            else:
                parts = line.split()
                if len(parts) >= 3:
                    words2.append(parts[0])
                    stags2.append(parts[2])
        if words2:
            sentences2.append(words2)
            seclabels2.append(stags2)

max_s = min(200, len(sentences2))
for i in range(max_s):
    tokens = sentences2[i]
    slabs = seclabels2[i]

    enc = tokenizer(tokens, is_split_into_words=True, return_tensors='pt',
                    truncation=True, max_length=256, padding='max_length')
    word_ids = enc.word_ids()

    with torch.no_grad():
        _, _, hidden = model(**enc)

    prev = None
    for j, wid in enumerate(word_ids):
        if wid is not None and wid != prev and wid < len(slabs):
            stag = slabs[wid]
            if stag != '0':
                sec_embeddings.append(hidden[0, j].numpy())
                sec_labels_list.append(stag)
            prev = wid

if len(sec_embeddings) > 50:
    emb_sec = np.array(sec_embeddings)
    perp = min(30, len(emb_sec) - 1)
    tsne_sec = TSNE(n_components=2, random_state=42, perplexity=perp)
    reduced_sec = tsne_sec.fit_transform(emb_sec)

    unique_secs = list(set(sec_labels_list))
    sec_colors = {'PERSONAL': '#ef4444', 'EMP_INFO': '#f59e0b', 'EMP_DESC': '#10b981',
                  'EDUCATION': '#3b82f6', 'SKILLS': '#8b5cf6'}

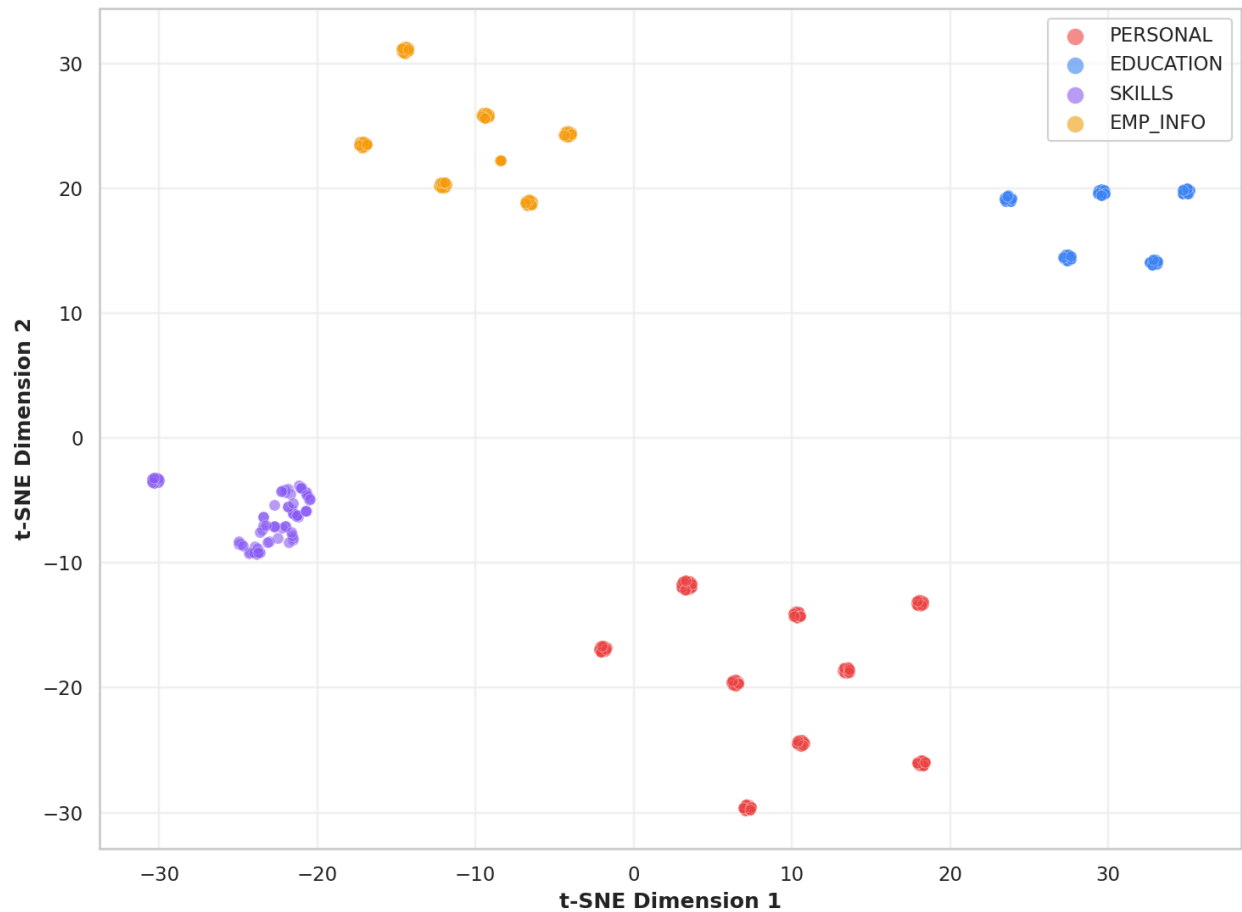
    fig, ax = plt.subplots(figsize=(10, 8))
    for sec in unique_secs:
        mask = [l == sec for l in sec_labels_list]
        points = reduced_sec[mask]
        ax.scatter(points[:, 0], points[:, 1], label=sec, alpha=0.6, s=40,
                  color=sec_colors.get(sec, '#94a3b8'), edgecolors='white', linewidth=0.3)

    ax.set_title('FIGURE 17. t-SNE of Fine-tuned XLM-RoBERTa Token Embeddings\n(Section Level, Validation Set)',
                fontweight='bold', fontsize=13, pad=15)
    ax.legend(fontsize=11, markerscale=1.5, frameon=True, fancybox=True)
    ax.set_xlabel('t-SNE Dimension 1', fontweight='bold')
    ax.set_ylabel('t-SNE Dimension 2', fontweight='bold')
    ax.grid(True, alpha=0.2)

    plt.tight_layout()
    plt.savefig('fig17_tsne_section.png', bbox_inches='tight', dpi=300)
    plt.show()
    print("✅ Saved: fig17_tsne_section.png")
else:
    print("⚠️ Not enough section tokens for t-SNE")

```

FIGURE 17. t-SNE of Fine-tuned XLM-RoBERTa Token Embeddings (Section Level, Validation Set)



```
!pip install adjustText -q

from adjustText import adjust_text

# Compute correlation between cluster compactness and F1 for each entity class
if len(all_embeddings) > 50:
    unique_ents = list(set(all_labels_list))
    emb_arr = np.array(all_embeddings)

    avg_distances = []
    f1_scores_ent = []
    ent_names = []

    for ent in unique_ents:
        mask = np.array([l == ent for l in all_labels_list])
        if mask.sum() < 3:
            continue

        ent_points = emb_arr[mask]
        centroid = ent_points.mean(axis=0)
        distances = np.sqrt(((ent_points - centroid)**2).sum(axis=1))
        avg_dist = distances.mean()

        # Get F1 for this entity from report
        f1_val = None
        for key, m in item_report.items():
            if isinstance(m, dict) and ent in key and 'f1-score' in m:
```